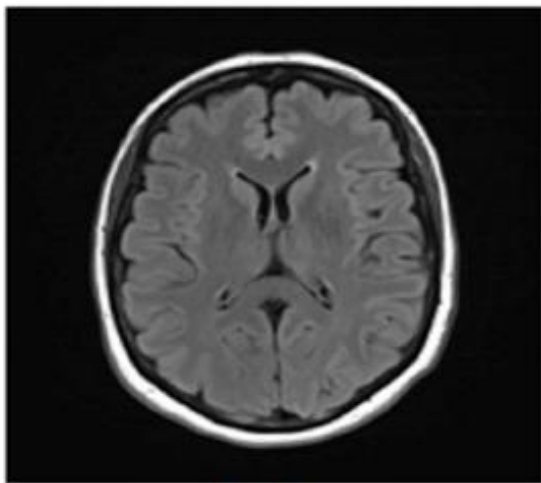
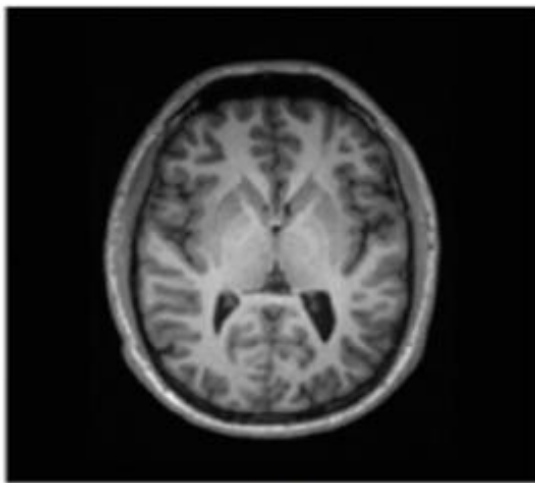


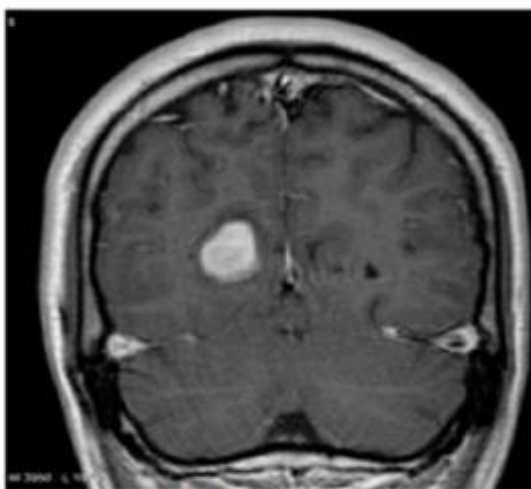
Brain Tumor Classification using Novel Area Attention Mechanism



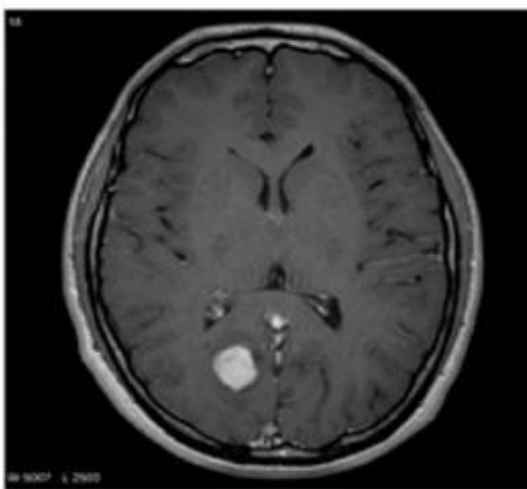
NO



NO



Yes



Yes

```
import numpy as np
import pandas as pd
import os
```

```
base_path = "/kaggle/input/brain-tumor-multimodal-image-ct-and-
mri/Dataset/Brain Tumor MRI images"
categories = ["Healthy", "Tumor"]
```

```
image_paths = []
labels = []
```

```
for category in categories:
```

```

category_path = os.path.join(base_path, category)
for image_name in os.listdir(category_path):
    image_path = os.path.join(category_path, image_name)
    image_paths.append(image_path)
    labels.append(category)

df = pd.DataFrame({
    "image_path": image_paths,
    "label": labels
})

df.head()

      image_path  label
0  /kaggle/input/brain-tumor-multimodal-image-ct-...  Healthy
1  /kaggle/input/brain-tumor-multimodal-image-ct-...  Healthy
2  /kaggle/input/brain-tumor-multimodal-image-ct-...  Healthy
3  /kaggle/input/brain-tumor-multimodal-image-ct-...  Healthy
4  /kaggle/input/brain-tumor-multimodal-image-ct-...  Healthy

df.tail()

      image_path  label
4995 /kaggle/input/brain-tumor-multimodal-image-ct-...  Tumor
4996 /kaggle/input/brain-tumor-multimodal-image-ct-...  Tumor
4997 /kaggle/input/brain-tumor-multimodal-image-ct-...  Tumor
4998 /kaggle/input/brain-tumor-multimodal-image-ct-...  Tumor
4999 /kaggle/input/brain-tumor-multimodal-image-ct-...  Tumor

df.shape

(5000, 2)

df.columns

Index(['image_path', 'label'], dtype='object')

df.duplicated().sum()

0

df.isnull().sum()

image_path    0
label         0
dtype: int64

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype

```

```

---  -----  -----  -----
0   image_path  5000 non-null  object
1   label       5000 non-null  object
dtypes: object(2)
memory usage: 78.2+ KB

df['label'].unique()

array(['Healthy', 'Tumor'], dtype=object)

df['label'].value_counts()

label
Tumor      3000
Healthy    2000
Name: count, dtype: int64

import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Tumor Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

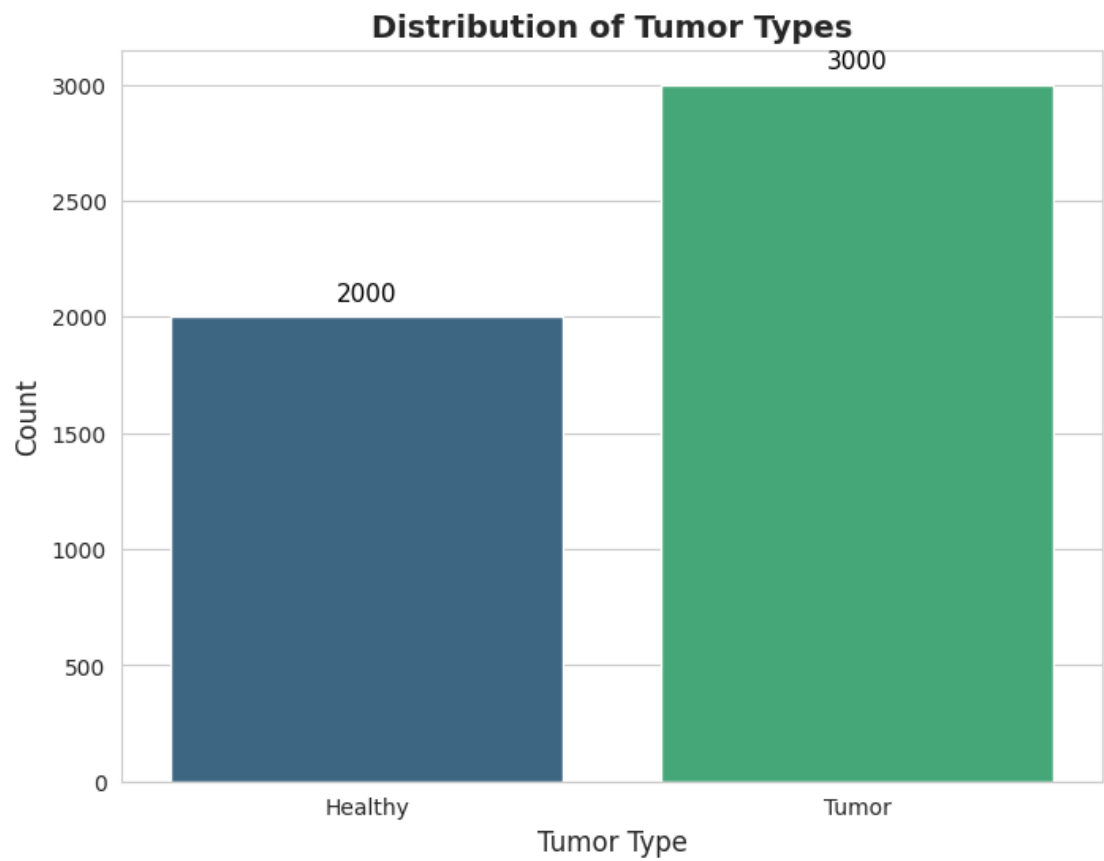
fig, ax = plt.subplots(figsize=(8, 6))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 12, 'weight':
'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

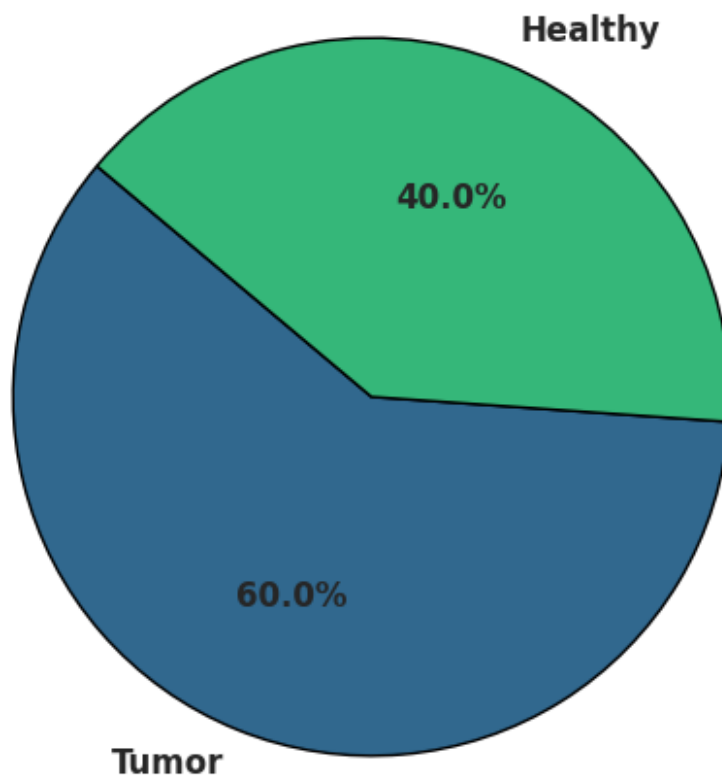
ax.set_title("Distribution of Tumor Types - Pie Chart", fontsize=14,
fontweight='bold')

```

```
plt.show()
```



Distribution of Tumor Types - Pie Chart



```
import cv2

num_images = 5

plt.figure(figsize=(15, 12))

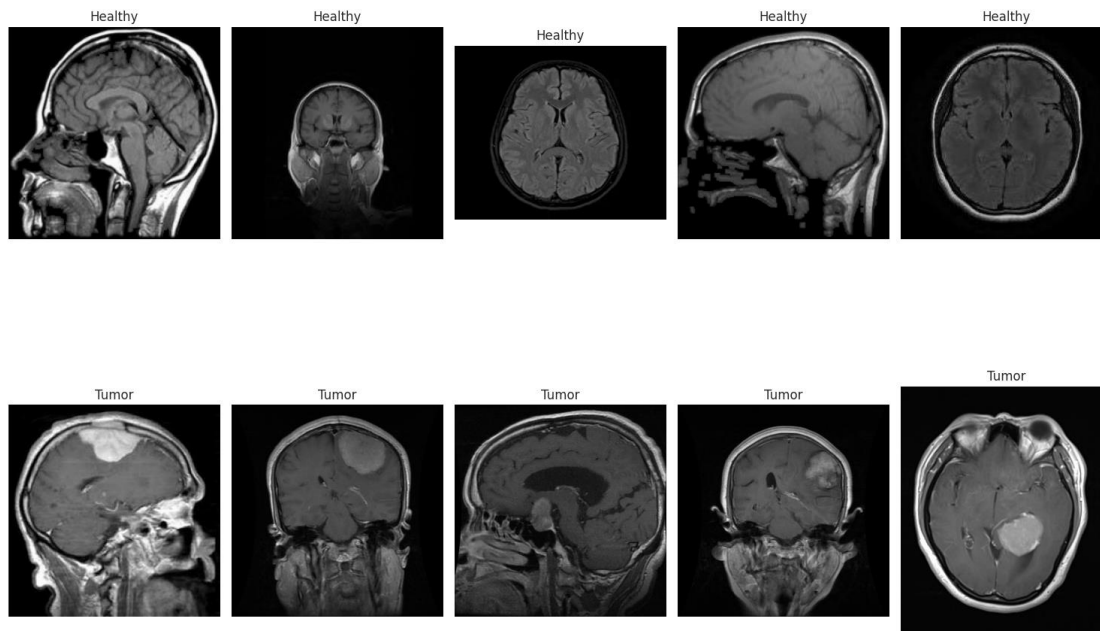
for i, category in enumerate(categories):
    category_images = df[df['label'] ==
category]['image_path'].iloc[:num_images]

    for j, img_path in enumerate(category_images):

        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        plt.subplot(len(categories), num_images, i * num_images + j + 1)
        plt.imshow(img)
        plt.axis('off')
        plt.title(category)
```

```
plt.tight_layout()
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder
```

```
label_encoder = LabelEncoder()
```

```
df['category_encoded'] = label_encoder.fit_transform(df['label'])
```

```
df = df[['image_path', 'category_encoded']]
```

```
from imblearn.over_sampling import RandomOverSampler
```

```
ros = RandomOverSampler(random_state=42)
```

```
X_resampled, y_resampled = ros.fit_resample(df[['image_path']],
df['category_encoded'])
```

```
df_resampled = pd.DataFrame(X_resampled, columns=['image_path'])
```

```
df_resampled['category_encoded'] = y_resampled
```

```
print("\nClass distribution after oversampling:")
```

```
print(df_resampled['category_encoded'].value_counts())
```

```
Class distribution after oversampling:
```

```
category_encoded
```

```
0    3000
```

```
1    3000
```

```
Name: count, dtype: int64
```

```
df_resampled
```

	image_path	category_encoded
0	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
1	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
2	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
3	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
4	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
...	...	...
5995	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
5996	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
5997	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
5998	/kaggle/input/brain-tumor-multimodal-image-ct-...	0
5999	/kaggle/input/brain-tumor-multimodal-image-ct-...	0

[6000 rows x 2 columns]

```
df_resampled['category_encoded'] =
df_resampled['category_encoded'].astype(str)
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Activation, Dropout, BatchNormalization
from tensorflow.keras import regularizers
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
print ('check')
```

check

```
train_df_new, temp_df_new = train_test_split(
    df_resampled,
    train_size=0.8,
    shuffle=True,
    random_state=42,
    stratify=df_resampled['category_encoded']
)
```

```
valid_df_new, test_df_new = train_test_split(
    temp_df_new,
    test_size=0.5,
    shuffle=True,
    random_state=42,
```

```

        stratify=temp_df_new['category_encoded']
    )

    batch_size = 16
    img_size = (224, 224)
    channels = 3
    img_shape = (img_size[0], img_size[1], channels)

    tr_gen = ImageDataGenerator(rescale=1./255)
    ts_gen = ImageDataGenerator(rescale=1./255)

    train_gen_new = tr_gen.flow_from_dataframe(
        train_df_new,
        x_col='image_path',
        y_col='category_encoded',
        target_size=img_size,
        class_mode='binary',
        color_mode='rgb',
        shuffle=True,
        batch_size=batch_size
    )

    valid_gen_new = ts_gen.flow_from_dataframe(
        valid_df_new,
        x_col='image_path',
        y_col='category_encoded',
        target_size=img_size,
        class_mode='binary',
        color_mode='rgb',
        shuffle=True,
        batch_size=batch_size
    )

    test_gen_new = ts_gen.flow_from_dataframe(
        test_df_new,
        x_col='image_path',
        y_col='category_encoded',
        target_size=img_size,
        class_mode='binary',
        color_mode='rgb',
        shuffle=False,
        batch_size=batch_size
    )

```

Found 4800 validated image filenames belonging to 2 classes.
 Found 600 validated image filenames belonging to 2 classes.
 Found 600 validated image filenames belonging to 2 classes.

```

import tensorflow as tf

print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))

Num GPUs Available:  2

gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print("GPU is set for TensorFlow")
    except RuntimeError as e:
        print(e)

```

GPU is set for TensorFlow

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from typing import Tuple, Optional

class AreaAttentionLayer(keras.layers.Layer):
    def __init__(self, reduction_ratio: int = 8, **kwargs):
        super(AreaAttentionLayer, self).__init__(**kwargs)
        self.reduction_ratio = reduction_ratio

        self.conv1 = layers.Conv2D(32, kernel_size=1, activation='relu')
        self.conv2 = layers.Conv2D(1, kernel_size=3, padding='same',
activation='sigmoid')
        self.bn = layers.BatchNormalization()

    def build(self, input_shape):
        super().build(input_shape)

    def call(self, inputs):
        batch_size = tf.shape(inputs)[0]
        height, width, channels = inputs.shape[1], inputs.shape[2],
inputs.shape[3]

        area_height = max(1, height // 4)
        area_width = max(1, width // 4)

        pad_height = (area_height - (height % area_height)) % area_height
        pad_width = (area_width - (width % area_width)) % area_width

        if pad_height > 0 or pad_width > 0:
            inputs_padded = tf.pad(inputs,
[[0, 0],
[0, pad_height],

```

```

                                [0, pad_width],
                                [0, 0]])
    new_height = height + pad_height
    new_width = width + pad_width
else:
    inputs_padded = inputs
    new_height, new_width = height, width

x = tf.reshape(
    inputs_padded,
    [batch_size, new_height // area_height, area_height,
     new_width // area_width, area_width, channels]
)
x = tf.transpose(x, [0, 1, 3, 2, 4, 5])
x = tf.reshape(
    x,
    [batch_size * (new_height // area_height) * (new_width //
area_width),
     area_height, area_width, channels]
)

attention = self.conv1(x)
attention = self.bn(attention)
attention = self.conv2(attention)

attention = tf.reshape(
    attention,
    [batch_size, new_height // area_height, new_width // area_width,
     area_height, area_width, 1]
)
attention = tf.transpose(attention, [0, 1, 3, 2, 4, 5])
attention = tf.reshape(attention, [batch_size, new_height, new_width,
1])

if pad_height > 0 or pad_width > 0:
    attention = attention[:, :height, :width, :]
    output = inputs * attention
else:
    output = inputs_padded * attention

return output

def get_config(self):
    config = super().get_config()
    config.update({
        'reduction_ratio': self.reduction_ratio
    })
    return config

```

```

class CNNBlock(keras.layers.Layer):
    def __init__(self, filters: int, use_attention: bool = True, **kwargs):
        super(CNNBlock, self).__init__(**kwargs)
        self.filters = filters
        self.use_attention = use_attention

        if self.use_attention:
            self.area_attention = AreaAttentionLayer()

        self.conv = keras.Sequential([
            layers.Conv2D(filters, kernel_size=3, padding='same',
use_bias=False),
            layers.BatchNormalization(),
            layers.ReLU(),
            layers.Conv2D(filters, kernel_size=3, padding='same',
use_bias=False),
            layers.BatchNormalization()
        ])

        self.shortcut = keras.Sequential()

    def build(self, input_shape):
        if self.filters != input_shape[-1]:
            self.shortcut = keras.Sequential([
                layers.Conv2D(self.filters, kernel_size=1, padding='same',
use_bias=False),
                layers.BatchNormalization()
            ])
        super().build(input_shape)

    def call(self, inputs):
        if self.use_attention:
            x = self.area_attention(inputs)
        else:
            x = inputs

        x = self.conv(x)

        shortcut = self.shortcut(inputs) if len(self.shortcut.layers) > 0
else inputs
        x += shortcut
        return tf.nn.relu(x)

    def get_config(self):
        config = super().get_config()
        config.update({
            'filters': self.filters,
            'use_attention': self.use_attention
        })

```

```

    return config

class RESNet(keras.Model):
    def __init__(self, num_classes: int = 2, dropout_rate: float = 0.3,
                  initial_filters: int = 32, **kwargs):
        super(RESNet, self).__init__(**kwargs)
        self.num_classes = num_classes
        self.dropout_rate = dropout_rate
        self.initial_filters = initial_filters

        self.stem = keras.Sequential([
            layers.Conv2D(self.initial_filters, kernel_size=7, strides=2,
                          padding='same', use_bias=False),
            layers.BatchNormalization(),
            layers.ReLU(),
            layers.MaxPooling2D(pool_size=3, strides=2, padding='same')
        ])

        self.blocks = [
            CNNBlock(self.initial_filters * (2**i), use_attention=True)
            for i in range(3)
        ]

        self.downsamples = [
            keras.Sequential([
                layers.Conv2D(self.initial_filters * (2**(i+1)),
                              kernel_size=3, strides=2, padding='same',
                              use_bias=False),
                layers.BatchNormalization(),
                layers.ReLU()
            ]) for i in range(2)
        ]

        self.final_attention = AreaAttentionLayer()
        self.global_pool = layers.GlobalAveragePooling2D()
        self.dropout = layers.Dropout(self.dropout_rate)
        self.fc = layers.Dense(1 if num_classes == 2 else num_classes,
                               activation='sigmoid' if num_classes == 2 else
                               'softmax')

    def call(self, inputs, training=None):
        x = self.stem(inputs)

        for i, block in enumerate(self.blocks):
            x = block(x)
            if i < len(self.downsamples):
                x = self.downsamples[i](x)
            if training:
                x = self.dropout(x)

```

```

        x = self.final_attention(x)
        x = self.global_pool(x)
        if training:
            x = self.dropout(x)
        return self.fc(x)

    def get_config(self):
        config = super().get_config()
        config.update({
            'num_classes': self.num_classes,
            'dropout_rate': self.dropout_rate,
            'initial_filters': self.initial_filters
        })
        return config

def build_model(input_shape: Tuple[int, int, int] = (224, 224, 3),
                num_classes: int = 2, learning_rate: float = 1e-3):
    model = RESNet(num_classes=num_classes)

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='binary_crossentropy' if num_classes == 2 else
'sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    model.build(input_shape=(None, *input_shape))
    return model

if __name__ == "__main__":
    model = build_model(input_shape=(224, 224, 3), num_classes=2)
    model.summary()

    history = model.fit(
        train_gen_new,
        validation_data=valid_gen_new,
        epochs=10,
        batch_size=16,
        verbose=1
    )

Epoch 1/10
300/300 ————— 34s 55ms/step - accuracy: 0.8082 - loss: 0.4489
- val_accuracy: 0.6367 - val_loss: 0.6965
Epoch 2/10
300/300 ————— 14s 46ms/step - accuracy: 0.8689 - loss: 0.3028
- val_accuracy: 0.7950 - val_loss: 0.4997
Epoch 3/10
300/300 ————— 14s 47ms/step - accuracy: 0.8866 - loss: 0.2777

```

```

- val_accuracy: 0.8767 - val_loss: 0.3058
Epoch 4/10
300/300 _____15s 48ms/step - accuracy: 0.9092 - loss: 0.2249
- val_accuracy: 0.5067 - val_loss: 3.3088
Epoch 5/10
300/300 _____14s 45ms/step - accuracy: 0.9321 - loss: 0.1809
- val_accuracy: 0.6950 - val_loss: 0.6864
Epoch 6/10
300/300 _____14s 45ms/step - accuracy: 0.9434 - loss: 0.1544
- val_accuracy: 0.9400 - val_loss: 0.1579
Epoch 7/10
300/300 _____14s 47ms/step - accuracy: 0.9411 - loss: 0.1568
- val_accuracy: 0.6550 - val_loss: 1.2377
Epoch 8/10
300/300 _____14s 47ms/step - accuracy: 0.9502 - loss: 0.1297
- val_accuracy: 0.8967 - val_loss: 0.2446
Epoch 9/10
300/300 _____14s 45ms/step - accuracy: 0.9575 - loss: 0.1081
- val_accuracy: 0.9433 - val_loss: 0.1433
Epoch 10/10
300/300 _____14s 46ms/step - accuracy: 0.9614 - loss: 0.0963
- val_accuracy: 0.8767 - val_loss: 0.3040

```

```

from sklearn.metrics import confusion_matrix, classification_report

```

```

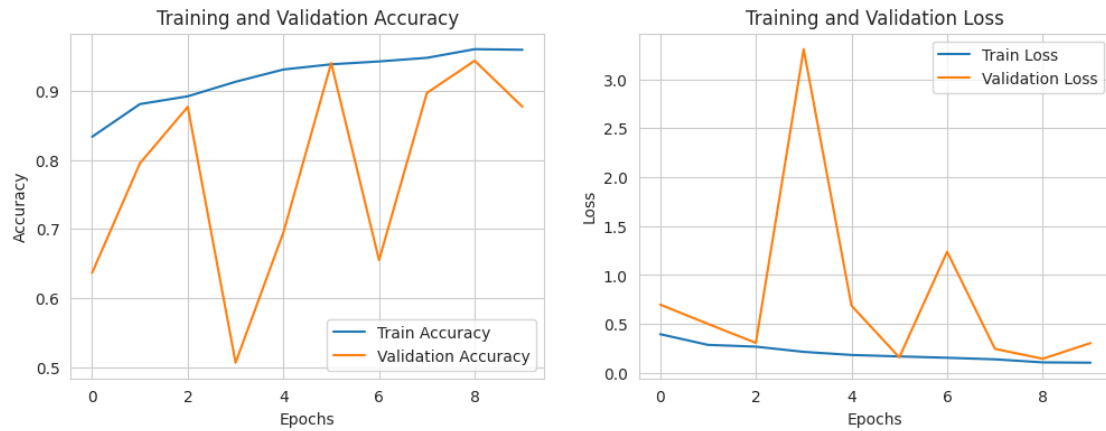
def plot_history(history):
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.title('Training and Validation Accuracy')

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.title('Training and Validation Loss')
    plt.show()

plot_history(history)

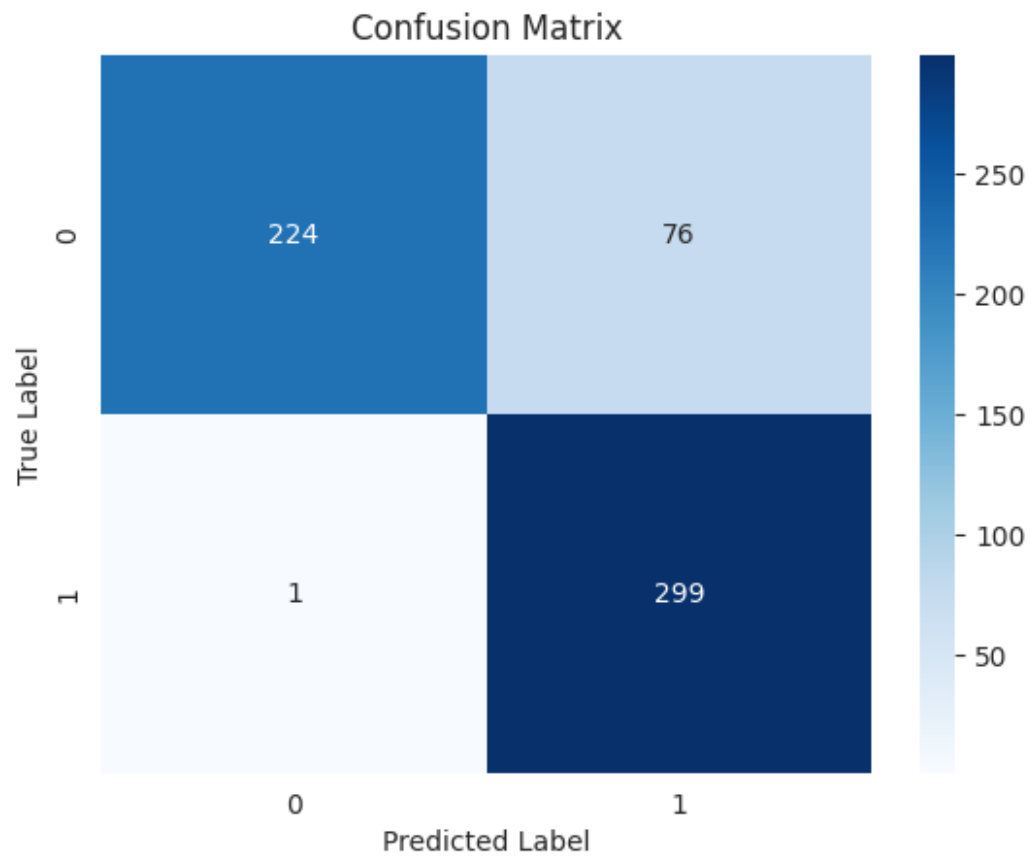
```



```
y_pred = model.predict(test_gen_new)
y_pred_classes = (y_pred > 0.5).astype(int).flatten()
y_true = test_gen_new.classes
```

38/38 ————— 4s 71ms/step

```
conf_matrix = confusion_matrix(y_true, y_pred_classes)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```



```
print('Classification Report:\n', classification_report(y_true,  
y_pred_classes))
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.75	0.85	300
1	0.80	1.00	0.89	300
accuracy			0.87	600
macro avg	0.90	0.87	0.87	600
weighted avg	0.90	0.87	0.87	600